

04 - Entity Framework Core

📅 09-09-2026 ⌚ Ongeveer 23 minuten 🗨️ Ongeveer 3381 woorden

Nu je weet hoe je een demo-project kunt aanmaken, opstarten en testen is het tijd om de fake database uit de API te vervangen door echte gegevens uit een PostgreSQL-database. Om de databaseverbinding te maken, gebruik je in deze cursus Entity Framework Core (Entity Framework Core) en optioneel Dapper.

Entity Framework is een **ORM (Object Relational Mapping) framework** dat C#-modellen omzet naar SQL-tabellen. Je project gebruikt bijvoorbeeld een string voor een tekstvariabele, en Entity Framework Core zorgt ervoor dat dit in de database wordt omgezet naar het juiste type, zoals text of varchar.

Met Entity Framework kun je de database en de tabellen volledig aanmaken vanuit je code. Dit principe heet *Code-First* en heeft als grote voordeel dat je zelf geen ingewikkelde SQL-scripts hoeft te schrijven om je database op te zetten.

Terugblik & Focus

In het vorige hoofdstuk hebben we de controller ontkoppeld van de harde datalogica door middel van het Repository Pattern en Interfaces. We leerden hoe de .NET Dependency Injection (DI) container automatisch de juiste objecten via de constructor aanlevert.

Nu we een losgekoppelde architectuur hebben, gaan we de tijdelijke lijst vervangen door een échte, persistente database-infrastructuur. We introduceren Entity Framework Core (Entity Framework Core) als onze Code-First ORM-motor en leggen de basis voor een hybride architectuur door Dapper te kunnen introduceren voor razendsnelle leesacties. We gebruiken Docker om een PostgreSQL-database en pgAdmin moeiteloos te containeriseren.

Info

De Code-First werkwijze: Je vertrekt altijd vanuit je C#-model(len). Entity Framework genereert op basis van deze modellen C#-scripts om tabellen aan te maken en/of bij te werken. Iedere keer als je een wijziging aanbrengt in je modellen, laat je een nieuw historisch script genereren. Dit heet een migratie.

Aan de hand van deze migraties update je de database-structuur. Als je nadien wijzigingen aanbrengt (bijvoorbeeld een eigenschap of extra Model toevoegen), maak je simpelweg een nieuwe migratie aan en update je opnieuw de database.

Het is ook mogelijk om Entity Framework te gebruiken als algemene querytaal. Wij schrijven onze queries met LINQ (**L**anguage **I**ntegrated **Q**uery). Bij het uitvoeren van de applicatie worden deze LINQ-queries door Entity Framework Core automatisch vertaald naar de specifieke SQL-dialecten van de gekozen databaseomgeving (zoals PostgreSQL, MS SQL Server, Oracle of SQLite). Zo kunnen we onze queries éénmaal schrijven en hergebruiken op verschillende dataplatformen.

Notitie

Waarom gebruiken we ook Dapper?

Als Entity Framework al het zware werk doet rondom databases, waarom hebben we Dapper dan nog nodig?

Entity Framework blinkt uit in het beheren van je database (de structuur, relaties en schrijfbewerkingen zoals Create, Update en Delete). Dapper is een zogenaamde Micro-ORM die uitblinkt in **pure snelheid**. Het zet SQL-queryresultaten vliegensvlug om naar C#-objecten zonder de extra overhead die een volwaardig ORM-framework zoals Entity Framework met zich meebrengt. In deze cursus combineren we het beste van twee werelden:

- Entity Framework voor de structuur (Code-First), migraties en complexe schrijfacties (Create/Update/Delete).
- Dapper voor het razendsnel ophalen van gegevens (Read) via directe SQL-queries.

Docker

Als je nog nooit met Docker hebt gewerkt, klinkt het misschien als zwarte magie, maar het is eigenlijk heel overzichtelijk.

Zie Docker als een soort 'mini-computer' (een container) die binnen jouw eigen computer draait. In plaats van dat je PostgreSQL helemaal moet installeren, configureren en services moet opstarten op je eigen besturingssysteem, start Docker een kant-en-klare database op met één druk op de knop. Als je de container stopt, is je computer weer helemaal 'schoon'.

Softwareontwikkelaars lopen vaak tegen de bekende frustratie op: "Op mijn computer werkt de code wel, maar op die van de collega of op de server niet." Dat komt doordat elke computer andere instellingen, besturingssystemen of geïnstalleerde softwareversies heeft. Docker is software die een programma én alles wat dat programma nodig heeft om te werken (zoals de database, specifieke taalversies en instellingen) samenvoegt in één afgesloten, draagbare doos: een container.



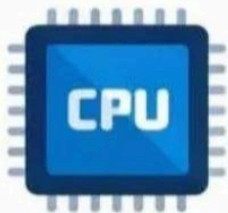
Docker haalt jouw eigen computer als variabele uit de vergelijking. Alles draait binnen de geïsoleerde container, waardoor de software op jouw laptop exact hetzelfde reageert als op een server in de cloud.

Info

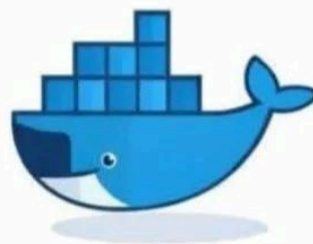
Containers worden vaak vergeleken met Virtuele Machines (zoals VirtualBox), maar er is een cruciaal verschil



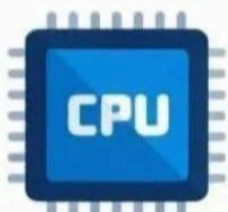
- **Virtuele Machine:** Simuleert een complete, zware computer inclusief een heel eigen besturingssysteem (zoals Windows of Linux). Dit kost gigantisch veel werkgeheugen (Ram geheugen, cpu cycles en schijfruimte) en start traag op.
- **Docker Container:** Maakt gebruik van de 'motor' (de kernel) van jouw eigen computer. Het isoleert alleen de software die je nodig hebt. Hierdoor starten containers in milliseconden op en gebruiken ze vrijwel geen geheugen. Alhoewel *vrijwel geen geheugen* een relatief begrip is.



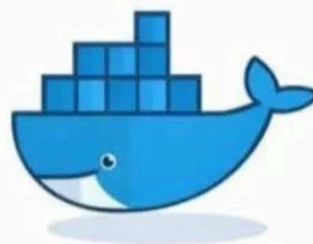
Docker, Docker



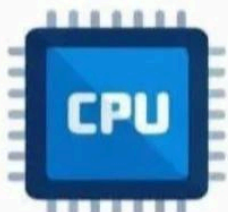
Yes papa



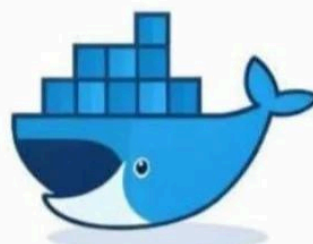
Eating RAM ?



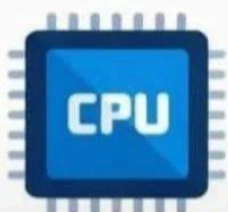
No papa



Telling lies ?



No papa



Open your mouth!

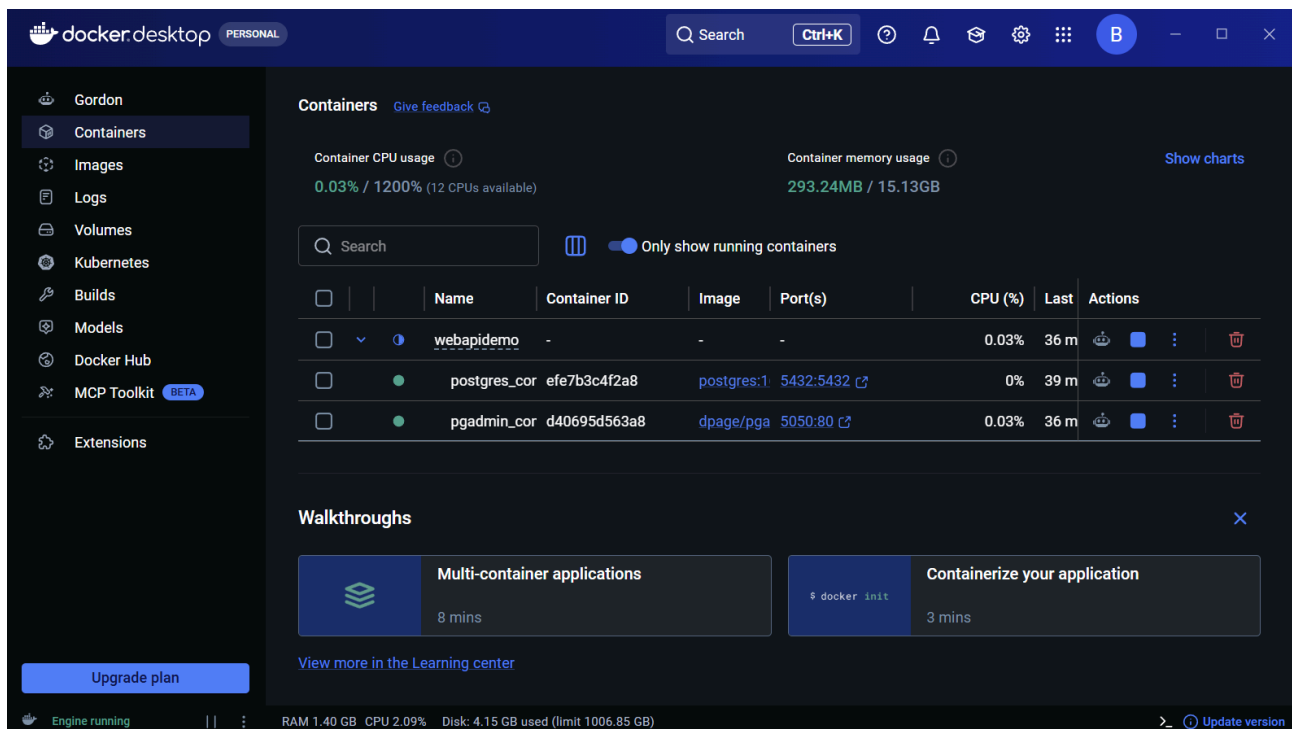


Process Name	Memory
com.docker.hyperkit	9.08 GB

Docker gebruiken

Om de nodige Docker Containers op te zetten gebruiken we `Docker Compose`. Dit is een simpel commando dat een tekstbestandje verwerkt waarin we het "recept" voor jouw software noteren. In dit geval zullen we een Image van een PostGres database gebruiken.

- Stap 1: Installeer Docker Desktop *als je dit nog niet gedaan hebt.*
 - Start Docker Desktop op. (Als er gevraagd wordt om een WSL 2-update op Windows, volg dan netjes de stappen op het scherm).
 - Zorg dat het icoontje linksonder ("Engine Running") in Docker Desktop groen kleurt (dit betekent dat Docker actief is).



- Stap 2: De container aanmaken (docker-compose.yml)
 - We gaan een configuratiebestand aanmaken. Dit bestand zorgt ervoor dat er niet alleen een PostgreSQL-database wordt opgestart, maar direct ook pgAdmin (de grafische beheeromgeving), zodat je meteen in je database kunt kijken.
 - Open je Web API-project (in Visual Studio of VS Code).
 - Maak in de hoofdmap van je project (naast je .sln of .csproj bestand) een nieuw bestand aan en noem het exact: docker-compose.yml.

- o Plak de onderstaande code in dat bestand:

```

1  services:
2      # Dienst 1: De PostgreSQL Database
3      postgres-db:
4          image: postgres:16-alpine # Een lichtgewicht, stabiele versie van
5      Postgres
6          container_name: postgres_container
7          restart: always
8          environment:
9              POSTGRES_USER: postgres
10             POSTGRES_PASSWORD: JeWachtwoordHier # Dit moet matchen met je
11 appsettings.json
12             POSTGRES_DB: WebAPIDemoDb
13         ports:
14             - "5432:5432" # Sluis poort 5432 van de container door naar jouw
15 localhost:5432
16         volumes:
17             - postgres_data:/var/lib/postgresql/data # Zorgt dat je data
18 bewaard blijft als de container stopt
19
20     # Dienst 2: pgAdmin (De online beheeromgeving)
21     pgadmin-tool:
22         image: dpage/pgadmin4
23         container_name: pgadmin_container
24         restart: always
25         environment:
26             PGADMIN_DEFAULT_EMAIL: student@opleiding.be # Je inlog-e-mail voor
27 pgAdmin
28             PGADMIN_DEFAULT_PASSWORD: Welkom01! # Je inlogwachtwoord voor
29 pgAdmin
30         ports:
31             - "8080:80" # Je opent pgAdmin via http://localhost:8080
32         depends_on:
33             - postgres-db
34
35     volumes:
36         postgres_data:

```

- Stap 3: De database starten

- o Open een terminal (PowerShell, Command Prompt, of de ingebouwde terminal van je IDE) en zorg dat je in de map staat waar de `docker-compose.yml` zich bevindt.

- Voer het volgende commando uit:

```
1 docker compose up -d bash
```

- (De -d staat voor 'detached', wat betekent dat de containers op de achtergrond blijven draaien en je terminal weer vrijkomt).
- Docker gaat nu de benodigde bestanden downloaden (dit gebeurt alleen de allereerste keer) en start de database op. In Docker Desktop zie je nu een groen mapje verschijnen met jouw projectnaam.
- Volg hierna de stappen in de Visual Studio Code of browser tab om je database te bekijken.

Handige Docker commando's voor later:

Containers stoppen (zonder data te verliezen):

```
1 docker compose down bash
```

Containers weer aanzetten:

```
1 docker compose up -d bash
```

Entity Framework en PostGres Sql installeren

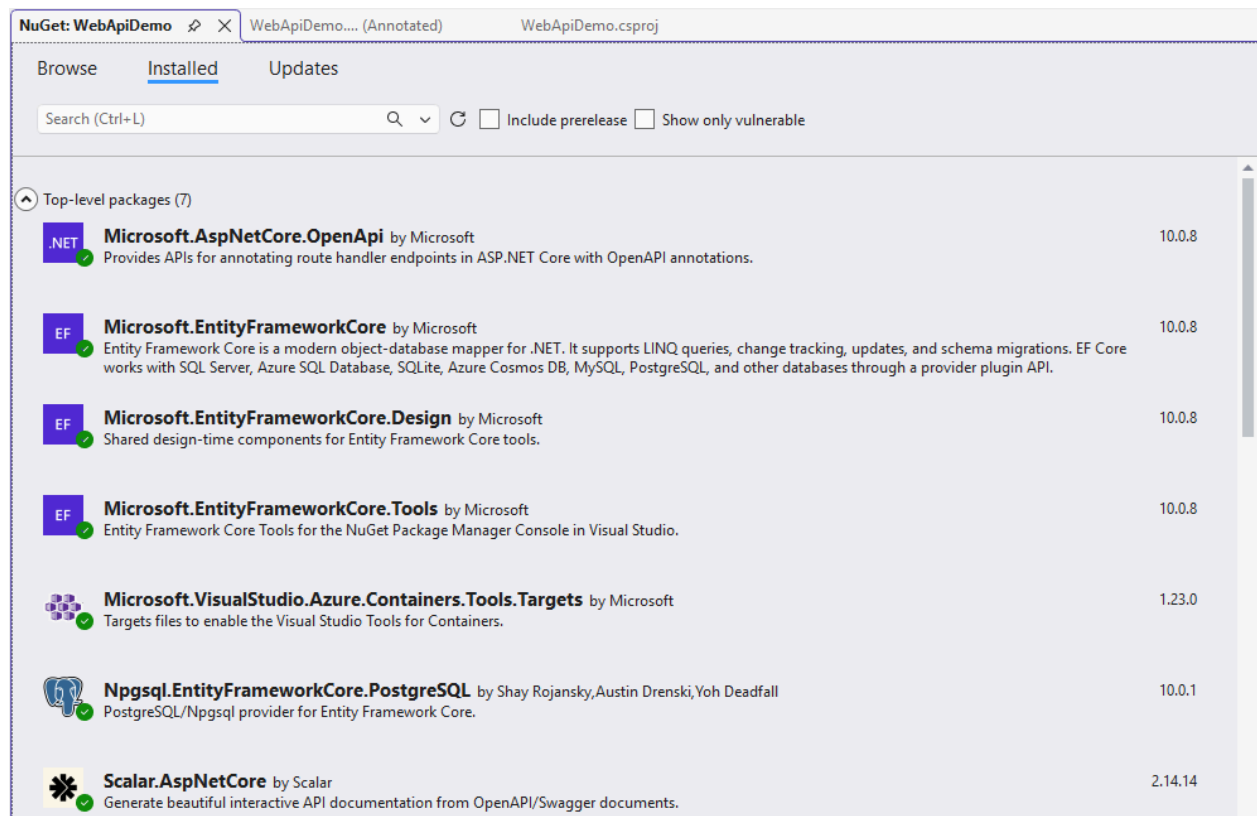
Voor je kunt beginnen met een database aan te maken, moet je nog wel eerst Entity Framework instellen. Het instellen van Entity Framework doe je in **5 stappen**:

1. Packages installeren
2. Databaseverbinding instellen in `appsettings.json`
3. `DbContext` aanmaken
4. `DbContext` registreren voor Dependency Injection in `Program.cs`
5. De initiële migratie aanmaken en uitvoeren

Stap 1: Packages installeren

Visual Studio

- Ga naar Tools -> NuGet Package Manager -> Manage NuGet Packages for Solution...
- Zoek en installeer:
 - **Microsoft.EntityFrameworkCore** (De core package)
 - **Npgsql.EntityFrameworkCore.PostgreSQL** (de PostgreSQL database-provider)
 - **Microsoft.EntityFrameworkCore.Design** (vereist om migraties aan te maken)
 - **Microsoft.EntityFrameworkCore.Tools** (vereist om migraties te kunnen beheren via de Package Manager Console)



Terminal

```
1 # Installeer de core package                                bash
2 dotnet add package Microsoft.EntityFrameworkCore
3
4 # Installeer de tools die nodig zijn voor het genereren van migraties
5 dotnet add package Microsoft.EntityFrameworkCore.Design
6
7
8
9
10
11
```

```
# Installeer de tools die nodig zijn voor het uitvoeren van migraties
dotnet add package Microsoft.EntityFrameworkCore.Tools

# Installeer de PostgreSQL-provider voor Entity Framework Core
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
```

Tips

Als je gebruik wilt maken van een andere database provider zoals Sql Server zal je een andere Package dan PostgreSQL moeten installeren. De rest van de packages blijven ongewijzigd.

Stap 2: Connection String instellen in appsettings.json

De applicatie moet weten waar de PostgreSQL-database zich bevindt en met welke inloggegevens er verbinding gemaakt moet worden. Deze configuratie slaan we op in het bestand `appsettings.json` onder de sectie `ConnectionStrings`.

In deze opleiding gebruik je Docker met een PostGres Sql database. Je geeft de naam op van de server (eventueel met poortnr), de naam van de database, gebruikersnaam wachtwoord en poort.

Daarnaast voorzien we ook nog alternatieve ConnectionString om een lokale DB op te stellen, moest je problemen ondervinden met Docker.

Pas je `appsettings.json` aan zodat deze er als volgt uitziet:

```
1  {
2      "Logging": {
3          "LogLevel": {
4              "Default": "Information",
5              "Microsoft.AspNetCore": "Warning"
6          }
7      },
8      "AllowedHosts": "*",
9      "ConnectionStrings": {
10
11
12
13
```

json

```
"LocalDBConnection": "Server=(localdb)\\mssqllocaldb;Database=WebAPIDemo";
"PostgresConnection":
"Host=localhost;Port=5432;Database=WebAPIDemoDb;Username=postgres;Password=
}
}
```

Stap 3: DbContext aanmaken

De `DbContext` is de centrale klasse die fungeert als de brug tussen jouw C#-code en de database. Hierin geef je aan welke C#-klassen (modellen) moeten worden omgezet naar database-tabellen via een `DbSet<T>`.

Maak een nieuwe klasse genaamd `WebAPIDemoContext.cs` aan in een nieuwe folder **Data**. Laat de klasse inheriten van `DbContext` en voeg voor elk model dat je wilt omzetten naar een tabel een **Property** van het type `DbSet<Model>` toe. Momenteel hebben we enkel een Laptop model dus voeg een nieuwe Property van het type `DbSet<Laptop>` toe en geef het de naam **Laptops**.

```
1  using Microsoft.EntityFrameworkCore;
2  using WebAPIDemo.Models;
3
4  namespace WebApiDemo.Data
5  {
6      public class WebAPIDemoContext : DbContext
7      {
8          public WebAPIDemoContext(DbContextOptions<WebAPIDemoContext>
9  options)
10         : base(options) { }
11
12         // Op basis van dit model zal een tabel Laptops gegenereerd
13         worden.
14         public DbSet<Laptop> Laptops { get; set; }
15     }
16 }
```

Tips

In deze klasse plaats je bovenaan de nodige using-statements naar de mappen **Models** en **Microsoft.EntityFrameworkCore**. Je kunt herhalende usings eventueel ook centraliseren in een `GlobalUsings.cs` bestand.

De constructor van `WebAPIDemoContext` accepteert een `DbContextOptions`-object en geeft dit door aan de basisklasse (`base(options)`). Via deze opties krijgt de context later de juiste database-provider en connectiestring mee vanuit `Program.cs`.

Data Finetunen met `OnModelCreating`

Soms komen de standaardconventies van C# niet helemaal overeen met de gewenste databasestructuur (bijvoorbeeld een string wordt standaard omgezet naar een onbeperkt text type). Door de methode `OnModelCreating` te overschrijven (`override`), kun je de tabel eigenschappen tot in detail configureren via de `Fluent API` .

```
1  using Microsoft.EntityFrameworkCore;
2  using WebAPIDemo.Models;
3
4  namespace WebAPIDemo.Data
5  {
6      public class WebAPIDemoContext : DbContext
7      {
8          public WebAPIDemoContext(DbContextOptions<WebAPIDemoContext>
9  options)
10             : base(options) { }
11
12             public DbSet<Laptop> Laptops { get; set; }
13
14             protected override void OnModelCreating(ModelBuilder
15 modelBuilder)
16             {
17                 // Altijd eerst de basis-methode aanroepen
18                 base.OnModelCreating(modelBuilder);
19
20                 modelBuilder.Entity<Laptop>(entity =>
21                 {
22                     // Specificeer expliciet de tabelnaam (optioneel)
23                     entity.ToTable("Laptops");
24
25                     // Beperk de lengte van strings om database-overhead te
26                     vermijden (wordt varchar(X))
27                     entity.Property(p =>
28 p.Merk).IsRequired().HasMaxLength(50);
29                     entity.Property(p =>
30 p.Processor).IsRequired().HasMaxLength(100);
31                     entity.Property(p =>
```

csharp

```
31     p.GPU).IsRequired().HasMaxLength(100);
32
33         entity.Property(p => p.RamInGB).IsRequired();
34
35         // Zorgt voor een numeric(18,2) in PostgreSQL voor
nauwkeurige geldbedragen
        entity.Property(p =>
p.Prijs).IsRequired().HasPrecision(18, 2);
    });
    }
}
```

Tips

Gebruik `override + spatie` om alle overschrijfbare methodes te zien en snel te laten genereren.

Stap 4: DbContext registreren voor Dependency Injection in Program.cs

Om ervoor te zorgen dat onze controllers en services toegang hebben tot de database, moeten we de `WebAPIDemoContext` registreren in de `Dependency Injection (DI)` container van de applicatie. We koppelen hier ook de PostgreSQL-provider aan de connectiestring die we in stap 2 hebben gedefinieerd.

Tips

Je hoeft enkel hier de `ConnectionString` aan te passen om een andere database te gebruiken. De rest van de code blijft ongewijzigd.

Open `Program.cs` en voeg de registratie toe voordat `builder.Build()` wordt aangeroepen:

```
1     using Microsoft.EntityFrameworkCore;
2     using WebAPIDemo.Data;
3
4     var builder = WebApplication.CreateBuilder(args);
5
6     // Haal de connection string op uit appsettings.json
```

csharp

```
7     string connectionString =
8     builder.Configuration.GetConnectionString("PostgresConnection");
9
10    // Registreer de DbContext met de PostgreSQL provider (Npgsql)
11    builder.Services.AddDbContext<WebAPIDemoContext>(options =>
12        options.UseNpgsql(connectionString));
13
14    // ... de rest van de configuratie ...
```

Je volledige `program.cs` zou er als volgt uit moeten zien:

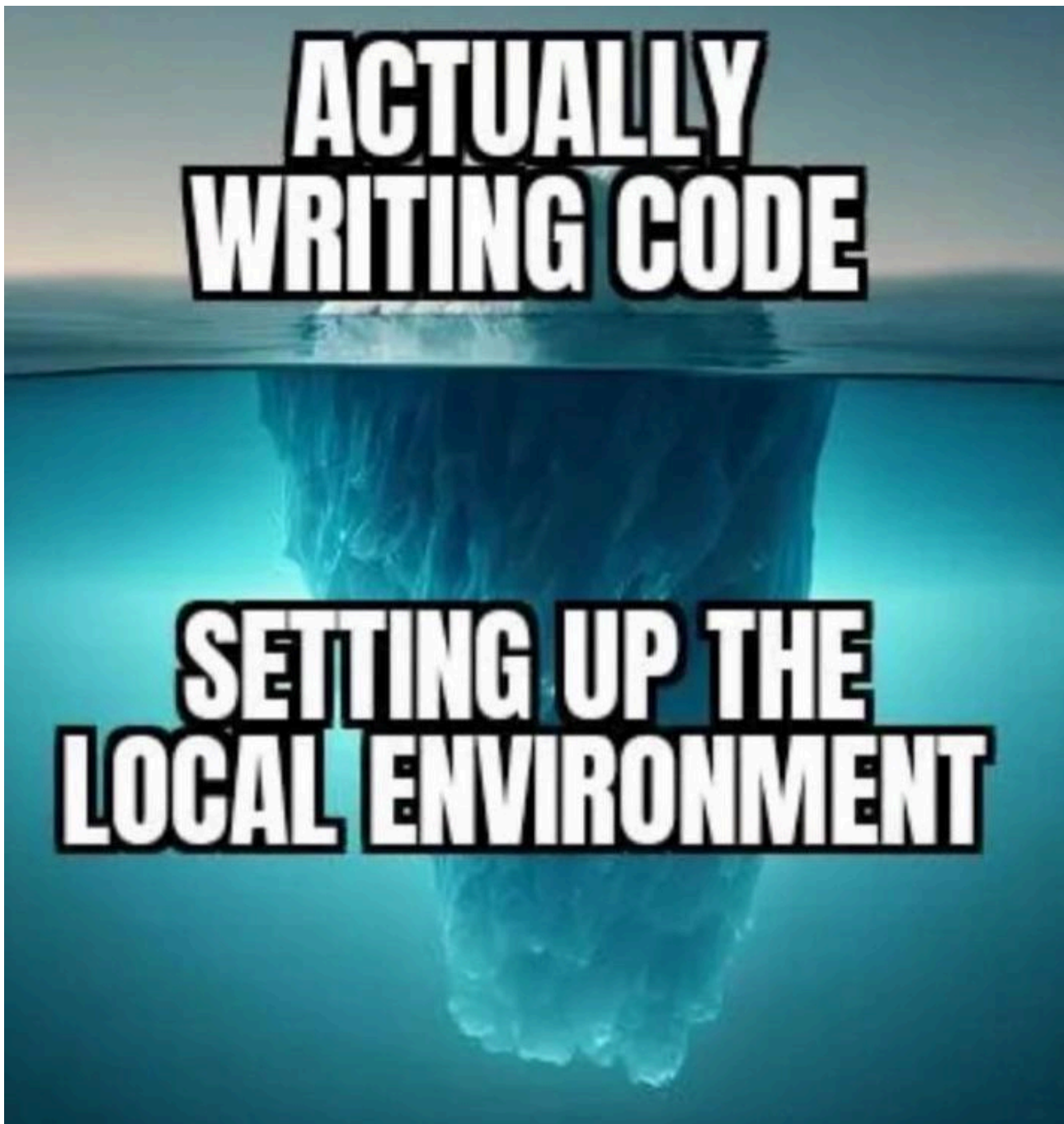
```
1     using Microsoft.EntityFrameworkCore;
2     using Scalar.AspNetCore;
3     using WebApiDemo.Data;
4     using WebAPIDemo.Repositories;
5
6     var builder = WebApplication.CreateBuilder(args);
7
8     // Add services to the container.
9
10    builder.Services.AddControllers();
11    // Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
12    builder.Services.AddOpenApi();
13
14    // Link Dependency Interface aan implementerende klasse
15    builder.Services.AddScoped<ILaptopRepository, InMemoryRepository>();
16
17    // Haal een connection string op uit appsettings.json
18    string connectionString =
19    builder.Configuration.GetConnectionString("PostgresConnection");
20
21    // Registreer de DbContext met de PostgreSQL provider (Npgsql)
22    builder.Services.AddDbContext<WebAPIDemoContext>(options =>
23        options.UseNpgsql(connectionString));
24
25    var app = builder.Build();
26
27    // Configure the HTTP request pipeline.
28    if (app.Environment.IsDevelopment())
29    {
30        app.MapOpenApi();
31        app.MapScalarApiReference(); // <-- Voeg dit toe voor de interactieve
    UI
```

csharp

```
32     }  
33  
34     app.UseHttpsRedirection();  
35  
36     app.UseAuthorization();  
37  
38     app.MapControllers();  
39  
    app.Run();
```

Notitie

👉 Zoals je ziet zijn er vrij veel stappen nodig om een database connectie vlot te laten verlopen. Op een examen mag je ervanuit gaan dat alle nodige packages, Connection Strings en Program configuratie beschreven in dit hoofdstuk reeds voorzien zullen zijn. Van jou zal enkel verwacht worden dat je een Database kan aanmaken op Docker, Migraties kunt aanmaken en uitvoeren, en data kan ophalen en wegschrijven.



Stap 5: Migratie(s) aanmaken

Als Entity Framework Core eenmaal correct is geconfigureerd, Docker runt en de verbinding met je database klaarstaat, kun je de feitelijke database gaan aanmaken of updaten. Als Docker nog niet runt, kan je later in dit hoofdstuk lezen hoe je dat doet. Omdat we de Code-First werkwijze hanteren, doen we dit in twee opeenvolgende stappen:

1. **Een migratie aanmaken** (Add-Migration): Entity Framework Core scant je C#-modellen en vergelijkt deze met een eerdere 'snapshot' (indien aanwezig). Vervolgens genereert het framework een C#-script in de map Migrations dat nauwkeurig beschrijft welke ta-

bellen of kolommen moeten worden toegevoegd, gewijzigd of verwijderd.

2. **De database updaten** (Update-Database): Nadat je bevestigd hebt dat Docker runt, kan je je database laten genereren en in de toekomst updaten door onderstaand commando in de Terminal te runnen. Entity Framework Core vertaalt het gegenereerde migratiescript automatisch naar de specifieke PostgreSQL-dialect-SQL en voert dit uit op de database die je target. **Als de database zelf nog niet bestaat, wordt deze op dit moment ook direct aangemaakt.**

Visual Studio (Package Manager Console)

Open de Package Manager Console via Tools -> NuGet Package Manager -> Package Manager Console en voer de volgende opdrachten uit:

```
1 # 1. Maak een migratiescript aan op basis van je C#-modellen in de DbContext en geef het een naam. bash
2 Add-Migration Initial_Create
3
4 # 2. Voer de migratie uit om de database en tabellen aan te maken in Docker.
5
6 # Later kan je dit commando uitvoeren om je database te updaten als er nieuwe Migraties werden aangemaakt.
Update-Database
```

Terminal / VS Code (.NET CLI)

Open de ingebouwde terminal in de hoofdmap van je project en voer de volgende .NET Entity Framework commando's uit:

```
1 # 1. Maak een migratiescript aan op basis van je C#-modellen in de DbContext en geef het een naam. bash
2 dotnet Entity Framework migrations add Initial_Create
3
4 # 2. Voer de migratie uit om de database en tabellen aan te maken in Docker.
5
6 # Later kan je dit commando uitvoeren om je database te updaten als er nieuwe Migraties werden aangemaakt.
dotnet Entity Framework database update
```

Hoe werkt zo'n migratiescript?

Na het uitvoeren van het `Add-Migration` commando verschijnt er een nieuw C#-bestand in de map `Migrations`.

```
1 reference
public partial class Initial_Create : Migration
{
    /// <inheritdoc />
    0 references
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "Laptops",
            columns: table => new
            {
                Id = table.Column<int>(type: "integer", nullable: false)
                    .Annotation("Npgsql:ValueGenerationStrategy", NpgsqlValueGenerationStrategy.IdentityByDefaultColumn),
                Merk = table.Column<string>(type: "character varying(50)", maxLength: 50, nullable: false),
                Processor = table.Column<string>(type: "character varying(100)", maxLength: 100, nullable: false),
                RamInGB = table.Column<int>(type: "tinyint", nullable: false),
                Prijs = table.Column<double>(type: "double precision", precision: 18, scale: 2, nullable: false),
                GPU = table.Column<string>(type: "character varying(100)", maxLength: 100, nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_Laptops", x => x.Id);
            });
    }

    /// <inheritdoc />
    0 references
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(
            name: "Laptops");
    }
}
```

Elk script bevat twee belangrijke methodes:

- De `Up` methode: Bevat de instructies om de database up-to-date te brengen. Dit wordt uitgevoerd wanneer je het `Update-Database` commando aanroept.
- De `Down` methode: Bevat de exacte instructies om de wijzigingen uit de `Up` methode weer ongedaan te maken. Dit is je **undo (rollback)** voor het geval er iets misgaat. Hier-voor gebruik je het `remove-migration` commando.

Notitie

Het commando `Remove-Migration` verwijdert uitsluitend het meest recente migratiebestand uit je C#-code, en alleen als deze nog niet op de database is toegepast. Is de database al bijgewerkt via `Update-Database`? Dan moet je de database eerst via de terminal terugrollen naar een eerdere versie voordat je het script mag verwijderen of de database in zijn geheel verwijderen.

Stap 6: Database verkennen

Idealiter gebruiken we `Docker` om omze PostGres database te gebruiken. Alternatief kan er ook gebruik gemaakt worden van een ingebouwde `mssql localdb` of ander alternatief. Op een examen kijken we enkel naar of je code en werking klopt, niet welk type DB je gebruikt hebt.

Visual Studio Code

- Klik links op het Extensions-icoon (of druk op Ctrl+Shift+X).
- Zoek naar PostgreSQL (de extensie van Chris Kolkman).
- Installeer de extensie. Er verschijnt nu een nieuw database-icoon in je linkerzijbalk.
- Klik op Add Connection (het plus-icoontje) en vul de gegevens van je Docker-container in:
 - **Host:** localhost
 - **Port:** 5432
 - **Database:** WebAPIDemoDb
 - **Username:** postgres
 - **Password:** JeWachtwoordHier

Je kunt nu direct in VS Code door je tabellen bladeren, data bekijken en SQL-queries uitvoeren.



A.I. : Gebruik een A.I. Client om een Sql script te genereren om een dertigtal laptops te genereren. Voeg deze toe aan je database zodat je een vergelijkbaar resultaat krijgt:

The screenshot shows the PostgreSQL Explorer interface in VS Code. On the left, a tree view shows the database structure: localhost > WebAPIDemoDb > public > Laptops. The Laptops table is selected, showing its columns: Id (integer), Merk (character varying(50)), Processor (character varying(100)), RamInGB (integer), Prijs (double precision), and GPU (character varying(100)). The main editor shows a SQL query: `SELECT * FROM "Laptops" LIMIT 1000;`. The results pane on the right shows 30 rows returned, displaying a list of laptops with their specifications.

		Id integer	Merk character varying	Processor character varying	RamInGB integer	Prijs double precision	GPU character varying
1	1		Apple	Apple M3	16	1829	M3 10-Core GPU
2	2		Apple	Apple M3 Pro	18	2549	M3 Pro 14-Core GPU
3	3		ASUS	AMD Ryzen 7 7735HS	16	1199	NVIDIA RTX 4060
4	4		ASUS	Intel Core i9-13900H	32	2199	NVIDIA RTX 4070
5	5		ASUS	AMD Ryzen 9 7940HS	32	2899	NVIDIA RTX 4080
6	6		Lenovo	AMD Ryzen 5 7520U	8	549	AMD Radeon 610M
7	7		Lenovo	Intel Core i7-1355U	16	949	Intel Iris Xe Graphics
8	8		Lenovo	AMD Ryzen 7 7840HS	16	1349	NVIDIA RTX 4060
9	9		HP	Intel Core i5-1235U	8	629	Intel Iris Xe Graphics
10	10		HP	Intel Core i7-13700H	16	1099	NVIDIA RTX 4050
11	11		HP	AMD Ryzen 5 5625U	16	699	AMD Radeon Graphics
12	12		Dell	Intel Core i7-1360P	16	1449	Intel Iris Xe Graphics
13	13		Dell	Intel Core i9-13900HK	32	2499	NVIDIA RTX 4070
14	14		Dell	Intel Core i5-1345U	16	1150	Intel Iris Xe Graphics
15	15		MSI	Intel Core i7-13620H	16	1049	NVIDIA RTX 4060
16	16		MSI	Intel Core i9-14900HX	64	4199	NVIDIA RTX 4090
17	17		MSI	AMD Ryzen 7 7735HS	16	899	NVIDIA RTX 4050

browser

Als je geen gebruik wilt of kunt maken van VsCode kan je je database ook rechtstreeks in een browser aanspreken.

pgAdmin is de officiële grafische beheeromgeving voor PostgreSQL. Deze wordt naast je database opgestart via Docker (meestal bereikbaar via <http://localhost:8080>).

- Open je browser en surf naar de [pgAdmin-pagina](#) .
- Log in met de inloggegevens uit je Docker-configuratie.
- Klik met de rechtermuisknop op Servers -> Register -> Server...
- Vul bij General een naam in (bijv. Docker Postgres).
- Vul bij het tabblad Connection de gegevens in:
 - **Host name/address:** postgres-db (of de specifieke servicenaam uit je docker-compose bestand)
 - **Port:** 5432
 - **Username:** postgres
 - **Password:** JeWachtwoordHier
 - Klik op Save.
 - Je kunt nu via de mappenstructuur aan de linkerkant je tabellen (Schemas -> public -> Tables) openen en data inzien.

Gouden regels voor Entity Framework Core Migraties:

- **Zorg dat Docker draait:** Je PostgreSQL Docker-container moet actief zijn wanneer je Update-Database uitvoert, anders kan Entity Framework Core de database niet bereiken.
- **Zinvolle naamgeving:** Geef je migraties altijd een duidelijke naam (bijv. InitialCreate, AddLaptopTable, AddProcessorToLaptop).
- **Controleer de code:** Voer je migraties niet blindelings uit. Werp altijd even een snelle blik in het gegenereerde Migrations-bestand om te zien of Entity Framework Core je modelwijzigingen correct heeft geïnterpreteerd.
- **Pas de database nooit rechtstreeks aan:** Werk altijd via migraties en pas nooit handmatig tabellen, kolommen of datatypes aan in je PostgreSQL-database zelf. Dit veroorzaakt onherstelbare conflicten bij de eerstvolgende migratie!

Wanneer je in de toekomst nieuwe modellen toevoegt of bestaande modellen uitbreidt, herhaal je deze cyclus: je registreert het model (indien nodig) als een DbSet in de databasecontext, genereert een nieuwe migratie en updatet de database.

Samenvatting

1. Entity Framework Core vs. Dapper

- **Entity Framework Core (Volwaardig ORM):** Beheert de databasestructuur (**Code-First**) en verwerkt complexe schrijfacties (*Create, Update, Delete*).
- **Dapper (Micro-ORM):** Puur gericht op maximale snelheid bij het ophalen van gegevens (*Read*) via directe SQL-queries.

2. Entity Framework Core in 5 Stappen

1. **Packages :** Installeer Entity Framework Core, Design, Tools en de PostgreSQL-provider (`Npgsql`).
2. **Connection String :** Locatie en inloggegevens van de database vastleggen in `appsettings.json` .

3. **DbContext** : De C#-klasse die fungeert als brug naar de database (bevat `DbSet<T>` en Fluent API-configuratie).
4. **Program.cs** : De `DbContext` registreren in de DI-container en koppelen aan de PostgreSQL-provider.
5. **Migraties** : Modelwijzigingen via de terminal omzetten in scripts en doorvoeren naar de database:
 - `dotnet Entity Framework migrations add <Naam>` (Script aanmaken met `Up` en `Down` methode)
 - `dotnet Entity Framework database update` (Wijzigingen daadwerkelijk toepassen)

3. Belangrijke Vuistregels

- **Docker** : De PostgreSQL-container moet actief draaien (`docker compose up -d`) voordat je migraties kunt uitvoeren.
- **Geen handmatige aanpassingen** : Wijzig de database **nooit** rechtstreeks via een externe tool; werk altijd via Entity Framework Core-migraties om conflicten te voorkomen.
- **Database bekijken :** Omdat de DB in Docker draait, inspecteer je de tabellen via een **VS Code extensie** (PostgreSQL) of **pgAdmin 4** op `localhost:5432` .

Laatst bijgewerkt: 09-09-2026, 08:52

Bijdragers: JorenSynaeeveTM